

Backtracking 2

12:47 PM



Content

01. Permutations 1
02. Permutations 2
03. Target sum subset
04. Knight's Tour

01. Given a string with distinct characters,
print all permutations

{abc} → a b c

3!

a c b

b a c

b c a

c a b

c b a

{ _ _ _ } { F F F }

a _ _

T F F

b _ _

F T F

c _ _

a b _

T T F

a c _

T F T

b a _

T T F

b c _

F T T

a b c

T T T

a c b

T T T

b a c

T T T

b c a

T T T

```
void permute ( ans[], idx, vis[], str )
```

```
    if ( idx == str.length ) {
```

```
        |   print (ans);
        |   return;
        |   }
```

```
    for ( i=0 ; i < n ; i++ )
```

```
        |   if ( vis[i] == false ) {
```

```
            |   vis[i] = true;
```

```
            |   ans[idx] = str[i];
```

```
            |   permute ( ans, idx+1, vis, str );
```

```
            |   vis[i] = false;
```

```
            |   ans[idx] = 0;
```

```
            |   }
```

```
        |   }
```

```
    }
```

TC: $O(n \times n!)$

SC: $O(n)$

* Approach 2

```
void permute ( ans[], idx, str )
{
    if ( idx == str.length ) {
        print (ans);
        return;
    }

    for ( i = 0 ; i < n ; i ++ )
    {
        if ( ans.contains ( str[i] ) ) continue;

        ans [idx] = str [i];
        permute ( ans , idx + 1 , vis, str );
        ans [idx] = 0;
    }
}
```

* Given a string with duplicate characters
print all unique permutations

$\{a, a, b\}$



a - -

a

b



a a -

a b -

a a -

a b -

b a -

b a -



a a b

a b a

a a b

a b a

b a a

b a a

Ans = a a b

a b a

b a a

```

void permute ( ans[], idx, vis[], str, hs<string> )
{
    if ( idx == str.length ) {
        hs.add ( ans );
        return;
    }

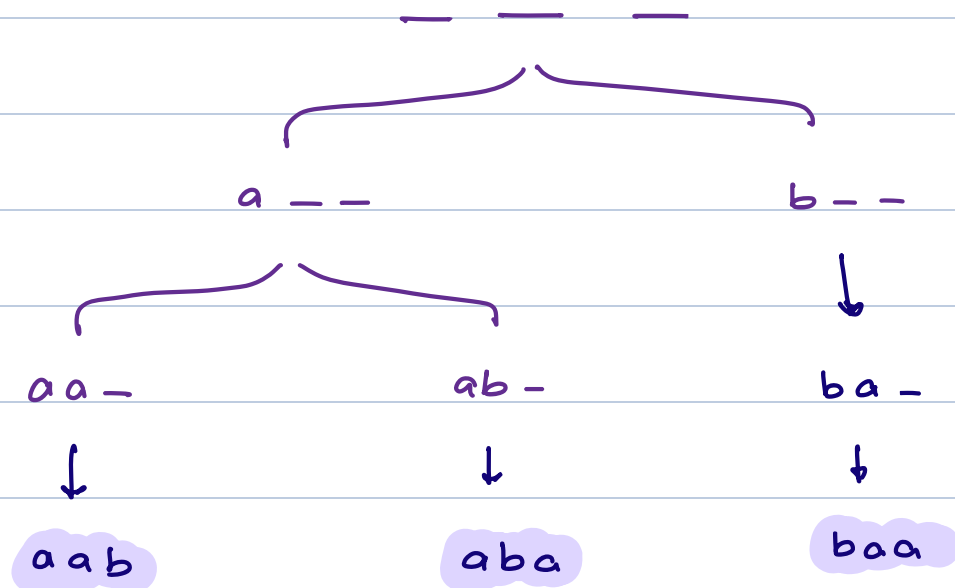
    for ( i = 0 ; i < n ; i ++ )
    {
        if ( vis[i] == false ) {
            vis[i] = true;
            ans[idx] = str[i];
            permute ( ans, idx + 1, vis, str, hs );
            vis[i] = false;
            ans[idx] = 0;
        }
    }
}

```

* Approach 2

{ a, a, b }

ele freq
 { a → ~~2~~ + 0
 { b → + 0 }



01. Build a hashmap with distinct characters & frequency

public void permute (smallans, n, hm, ans) ↖ str.length

```

if (smallans.size() == n) {
    ans.add(new AL<>(smallans));
    return;
}

```

```

for (int key : hm.keySet())

```

```

    int count = hm.get(key)

```

```

    if (count == 0) continue;

```

```

    smallans.add(key);

```

```

    hm.put(key, count - 1);

```

```
permute (smallans, n, hm, ans):
```

```
smallans.remove (smallans.size()-1);
```

```
hm.put (key, count);
```

3

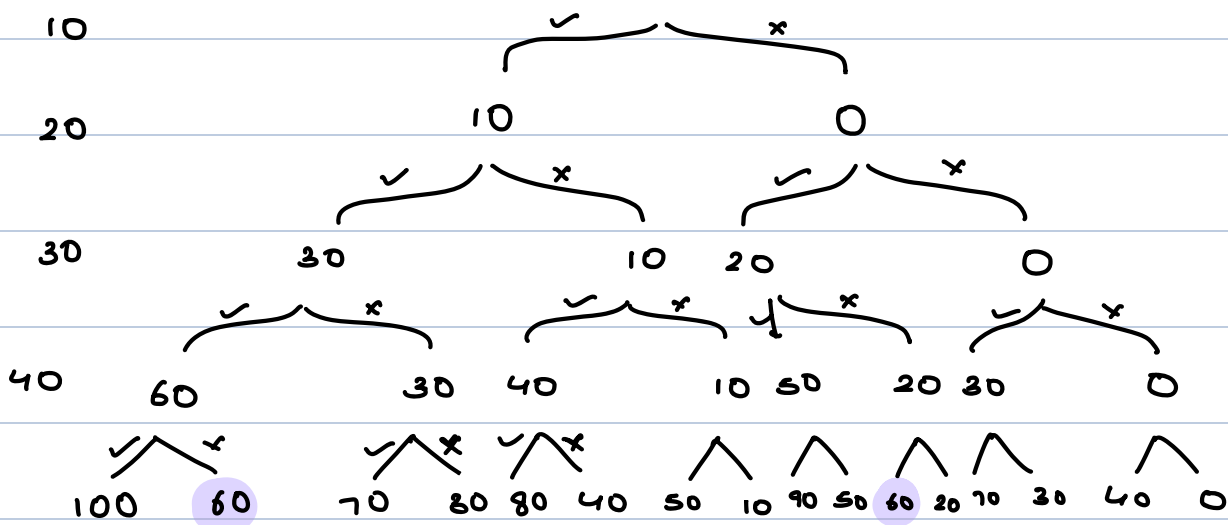
3

* Target sum subset

Given an array of non-negative numbers & a value tar, the task is to print all the subsets which are equal to given tar

$A[] = \{10, 20, 30, 40\}$ $tar = 60$

Ans = $\begin{cases} 10, 20, 30 \\ 20, 40 \end{cases}$



```

1  import java.util.*;
2  class Main {
3      public static void main(String args[]) {
4          // Your code goes here
5          int A[]={10,20,30};
6          print(A,0,new ArrayList<>(),0,30);
7      }
8      public static void print(int[]A,int idx,ArrayList<Integer>ans,int sum,int tar)
9      {
10         if(idx==A.length){
11             if(sum==tar){
12                 for(int i:ans)System.out.print(i+" ");
13             }
14             return;
15         }
16         ✓ans.add(A[idx]);
17         print(A,idx+1,ans,sum+A[idx],tar);
18         ✓ans.remove(ans.size()-1);
19         print(A,idx+1,ans,sum,tar);
20     }
}

```

Handwritten annotations:

- Line 6: `0` is labeled `idx`, `0` is labeled `sum`, and `30` is labeled `tar`.
- Line 16: `sum+A[idx]` is underlined, and the word `pick` is written next to it.
- Line 18: `print(A,idx+1,ans,sum,tar);` is labeled `not pick`.

* Knight's tour

Given a $N \times N$ chessboard with a knight initially standing on x ' row & y ' column, the task is to print all possible paths such that knight must visit each square only once.

